

Chapter 11: Database Handling

11.1. Introduction

A database is an organized collection of data. There are many different strategies for organizing data to facilitate easy access and manipulation. A database management system (DBMS) provides mechanisms for **storing**, **organizing**, **retrieving** and **modifying data** for many users. Database management systems allow for the access and storage of data without concern for the internal representation of data. Some popular relational database management systems (RDBMSs) are Microsoft SQL Server, Oracle, Sybase, IBM DB2, Informix, PostgreSQL and MySQL. The JDK now comes with a pure-Java RDBMS called Java DB-Oracles's version of Apache Derby.

Java programs communicate with databases and manipulate their data using the *Java Database Connectivity (JDBC)* API. A *JDBC* driver enables Java applications to connect to a database in a particular DBMS and allows us to manipulate that database using the *JDBC API*. Most popular database management systems now provide *JDBC drivers*. There are also many third-party JDBC drivers available.

The JDBC consists of two layers. The top layer is the *JDBC API*. This API communicates with the *JDBC manager driver API*, sending it the various SQL statements. The manager should communicate with the various third-party drivers that actually connect to the database and return the information from the query or perform the action specified by the query.

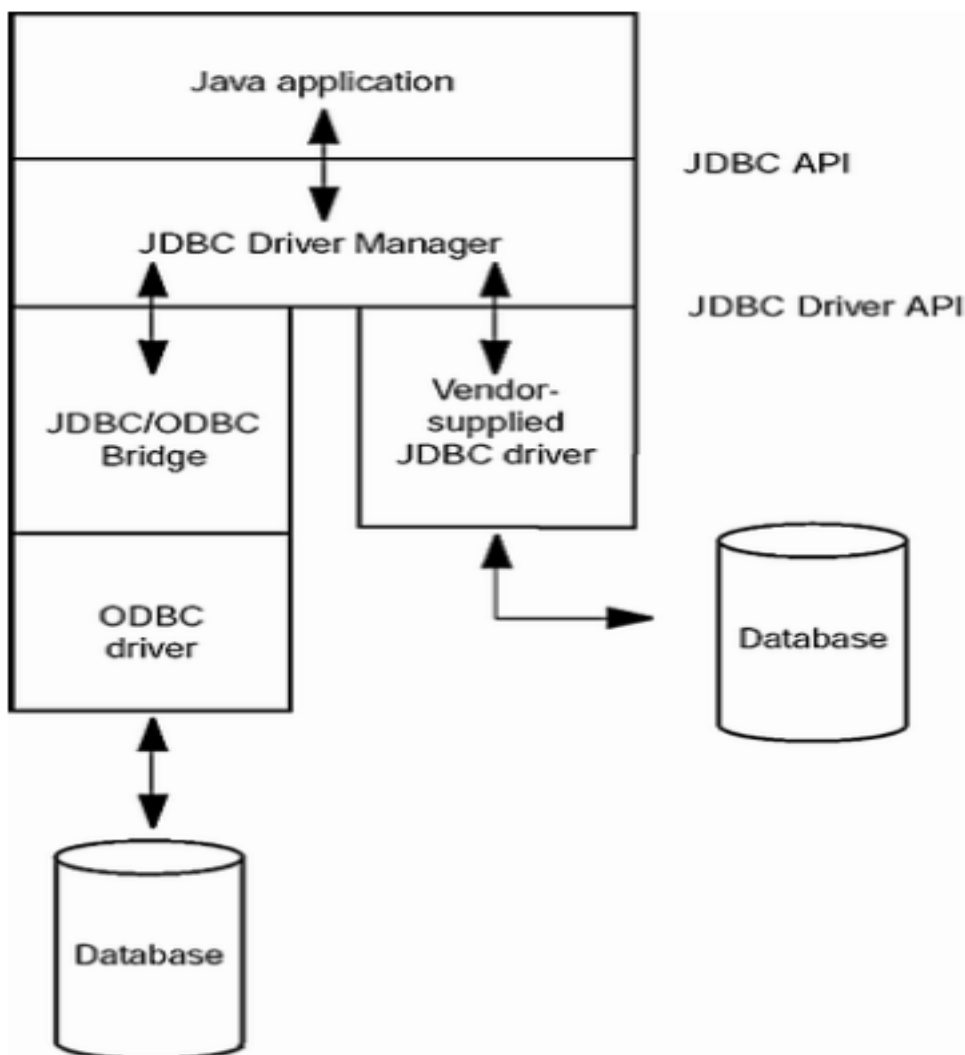


Fig: JDBC-to-database communication path

11.2. Classification of JDBC Drivers

JDBC drivers are classified into the following types

A type 1 driver translates JDBC to ODBC and relies on an ODBC driver to communicate with the database. Sun includes one such driver, the JDBC/ODBC Bridge, with the JDK. However, the bridge requires deployment and proper configuration of an ODBC driver. When JDBC was first released, the bridge was handy for testing, but it was never intended for production use.

A type 2 driver is a driver, written partly in the Java programming language and partly in native code, that communicates with the client API of a database. When we use such a driver, we must install some platform-specific code in addition to a Java library.

A type 3 driver is a pure Java client library that uses a database-independent protocol to communicate database requests to a server component, which then translates the requests into a database-specific protocol. The client library is independent of the actual database, thus simplifying deployment.

A type 4 driver is a pure Java library that translates JDBC requests directly to a database-specific protocol.

Most database vendors supply either a type 3 or type 4 drivers with their database. Furthermore, a number of third-party companies specialize in producing drivers with better standards conformance, support for more platforms, better performance, or, in some cases, simply better reliability than the drivers that are provided by the database vendors.

In summary, the ultimate goal of the JDBC is to make possible the following:

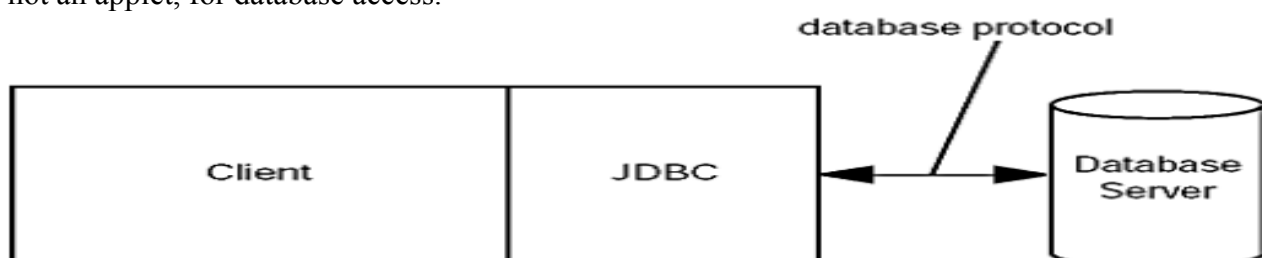
- Programmers can write applications in the Java programming language to access any database, using standard SQL statements or even specialized extensions of SQL-while still following Java language conventions.
- Database vendors and database tool vendors can supply the low-level drivers. Thus, they can optimize their drivers for their specific products.

11.3. Typical Uses of JDBC

We can use JDBC in both applications and applets. In an applet, all the normal security restrictions apply. By default, the security manager assumes that all applets written in the Java programming language are untrusted.

In particular, applets that use JDBC are only able to open a database connection to the server from which they are downloaded. That means the Web server and the database server must be the same machine, which is not a typical setup. Of course, the Web server can have a proxy service that routes database traffic to another machine. With signed applets, this restriction can be loosened.

Applications, on the other hand, have complete freedom to access remote database servers. If we implement a traditional client/server program, it probably makes more sense to use an application, not an applet, for database access.



However, the world is moving away from client/server and toward a "three-tier model" or even more

advanced "n-tier models." In the three-tier model, the client does not make database calls. Instead, it calls on a middleware layer on the server that in turn makes the database queries. The three-tier model has a couple of advantages. It separates **visual presentation** (on the client) from the **business logic** (in the middle tier) and the raw data (in the database). Therefore, it becomes possible to access the same data and the same business rules from multiple clients, such as a Java application or applet or a web form.

Communication between the client and middle tier can occur through HTTP (when we use a web browser as the client), RMI (when we use an application or applet), or another mechanism. *JDBC is used to manage the communication between the middle tier and the back-end database.* Following figure shows the basic architecture.

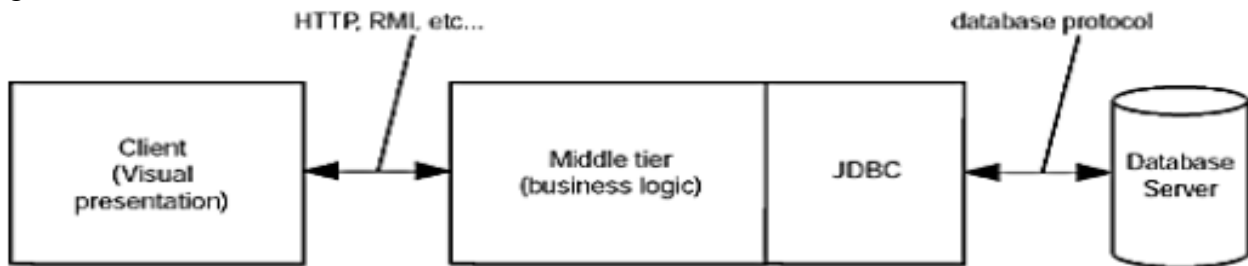


Fig: 3-tier Application

SQL is the industry-standard approach to accessing relational databases. JDBC supports SQL, enabling developers to use a wide range of database formats without knowing the specifics of the underlying database. JDBC also supports the use of database queries specific to a database format.

The JDBC class library's approach to accessing databases with SQL is comparable to existing database-development techniques, so interacting with an SQL database by using JDBC isn't much different than using traditional database tools. Java programmers who already have some database experience can hit the ground running with JDBC. The JDBC library includes classes for each of the tasks commonly associated with database usage:

- Making a connection to a database
- Creating a statement using SQL
- Executing that SQL query in the database
- Viewing the resulting records

These JDBC classes are all part of the *java.sql* package.

11.4. Database Drivers

Java programs that use JDBC classes can follow the familiar programming model of issuing SQL statements and processing the resulting data. The format of the database and the platform it was prepared on don't matter.

A driver manager makes the platform and database independence possible. The classes of the *JDBC class library are largely dependent on driver managers*, which keep track of the drivers required to access database records. We'll need a different driver for each database format that's used in a program, and sometimes we might need several drivers for versions of the same format. *Java DB includes its own driver. JDBC also includes a driver that bridges JDBC and another database-connectivity standard, ODBC.*

The JDBC-ODBC bridge allows JDBC drivers to be used as ODBC drivers by converting JDBC method calls into ODBC function calls.

Using the JDBC-ODBC bridge requires three things:

- The JDBC-ODBC bridge driver included with Java: *sun.jdbc.odbc.JdbcOdbcDriver*
- An ODBC driver
- An ODBC data source that has been associated with the driver using software such as the ODBC Data Source Administrator.

11.5. Manipulating Databases with JDBC

11.5.1. Connecting to and Querying a Database

The following section of program performs a simple query on the *books* database that retrieves the entire *Authors* table and displays the data. The program illustrates *connecting to the database, querying the database and processing the result*. The discussion that follows presents the key JDBC aspects of the program.

DisplayAuthors.java

// Displaying the contents of the authors table.

```
import java.sql.Connection;
import java.sql.Statement;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.ResultSetMetaData;
import java.sql.SQLException;

public class DisplayAuthors
{
    // database URL
    static final String DATABASE_URL = "jdbc:mysql://localhost:3306/books";

    // launch the application
    public static void main( String args[] )
    {
        Connection connection = null; // manages connection
        Statement statement = null; // query statement
        ResultSet resultSet = null; // manages results

        // connect to database books and query database
        try
        {
            // establish connection to database
            connection = DriverManager.getConnection(
                DATABASE_URL, "root", "" );

            // create Statement for querying database
            statement = connection.createStatement();

            // query database
            resultSet = statement.executeQuery(
                "SELECT authorID, firstName, lastName FROM authors");

            // process query results
            ResultSetMetaData metaData = resultSet.getMetaData();
            int numberOfColumns = metaData.getColumnCount();
            System.out.println( "Authors Table of Books Database:\n" );

            for ( int i = 1; i <= numberOfColumns; i++ )
                System.out.printf( "%-8s\t", metaData.getColumnName( i ) );
            System.out.println();

            while ( resultSet.next() )
            {
                for ( int i = 1; i <= numberOfColumns; i++ )
                    System.out.printf( "%-8s\t", resultSet.getObject( i ) );
                System.out.println();
            } // end while
        }
    }
}
```

```

    } // end try
    catch ( SQLException sqlException )
    {
        sqlException.printStackTrace();
    } // end catch
    finally // ensure resultSet, statement and connection are closed
    {
        try
        {
            resultSet.close();
            statement.close();
            connection.close();
        } // end try
        catch ( Exception exception )
        {
            exception.printStackTrace();
        } // end catch
    } // end finally
} // end main
} // end class DisplayAuthors

```

In this program there is an import the JDBC interfaces and classes from package **java.sql** is used and the declaration of a string constant for the *database URL*. This identifies the name of the database to connect to, as well as information about the *protocol* used by the JDBC driver. Method *main* connects to the *books* database, queries the database, displays the result of the query and closes the database connection.

11.5.2. Connecting to the Database

In an above program `connection=DriverManager.getConnection(DATABASE_URL,"root","")`; create a *Connection* object (package *java.sql*) referenced by *connection*. An object that implements interface **Connection** manages the connection between the *Java program and the database*. Connection objects enable programs to create SQL statements that manipulate databases. The program initializes connection with the result of a call to static method **getConnection** of class **DriverManager** (package *java.sql*), which attempts to connect to the database specified by its URL. Method *getConnection()* takes three arguments-a String that specifies the *database URL*, a String that specifies the *username* and a String that specifies the *password*. The URL locates the database (possibly on a network or in the local file system of the computer). The URL ***jdbc:mysql://localhost:3306/books*** specifies the protocol for communication (***jdbc***), the sub protocol for communication (***mysql***) and the location of the database (***//localhost:3306/books***, where *localhost* is the host running the MySQL server, *3306* is default *mysql* running port and ***books*** is the database name). The sub protocol *mysql* indicates that the program uses a MySQL-specific sub protocol to connect to the MySQL database. If the *DriverManager* cannot connect to the database, method *getConnection* throws a *SQLException* (package *java.sql*). Following Figure lists the JDBC driver names and database URL formats of several popular RDBMSs.

RDBMS	Database URL format
MySQL	<code>jdbc:mysql://hostname:portNumber/databaseName</code>
ORACLE	<code>jdbc:oracle:thin:@hostname:portNumber:databaseName</code>
DB2	<code>jdbc:db2:hostname:portNumber/databaseName</code>
PostgreSQL	<code>jdbc:postgresql://hostname:portNumber/databaseName</code>
Java DB/Apache Derby	<code>jdbc:derby:databaseName (embedded)</code> <code>jdbc:derby://hostname:portNumber/databaseName (network)</code>
Microsoft SQL Server	<code>jdbc:sqlserver://hostname:portNumber;databaseName=databaseName</code>
Sybase	<code>jdbc:sybase:Tds:hostname:portNumber/databaseName</code>

Fig: Popular JDBC database URL formats

11.5.3. Creating a *Statement* for Executing Queries

A *Connection* method ***createStatement*** to obtain an object that implements interface ***Statement*** (package *java.sql*). The program uses the ***Statement*** object to submit SQL statements to the database.

Type of Statements

There are 3 different kinds of statements.

I) *Statement*: Used to implement simple SQL statements with no parameters.

Eg: `statement = connection.createStatement();`

II) *PreparedStatement*: (Extends *Statement*.) Used for precompiling SQL statements that might contain input parameters. Eg:

```
PreparedStatement pstmt = null;
String SQL = " SELECT authorID, firstName, lastName FROM authors ";
pstmt = conn.prepareStatement(SQL);
```

III) *CallableStatement*: (Extends *PreparedStatement*.) Used to execute stored procedures that may contain both input and output parameters.

Suppose we have a store procedure *getAuthor* in MySql to retrieve authors.

```
DELIMITER $$
DROP PROCEDURE IF EXISTS getAuthor $$
CREATE PROCEDURE getAuthor
    (OUT authorID int, OUT FirstName VARCHAR(255), OUT LastName
    VARCHAR(255))
BEGIN
    SELECT authorID, firstName, lastName FROM authors;
END $$
```

In order to call those store procedure in program.

```
CallableStatement cs = null;
cs = this.conn.prepareCall("{call getAuthor()}");
ResultSet rs = cs.executeQuery();
```

Three types of parameters exist: IN, OUT, and INOUT

Parameter	Description
IN	A parameter whose value is unknown when the SQL statement is created. We bind values to IN parameters with the setXXX() methods.
OUT	A parameter whose value is supplied by the SQL statement it returns. We retrieve values from the OUT parameters with the getXXX() methods.
INOUT	A parameter that provides both input and output values. We bind variables with the setXXX() methods and retrieve values with the getXXX() methods.

11.5.4. Executing a Query

In above program use the *Statement* object's ***executeQuery*** method to submit a query that selects all the author information from table *Authors*. This method returns an object that implements interface ***ResultSet*** and contains the query results. The ***ResultSet*** methods enable the program to manipulate the query result.

- ***execute***: Returns true if the first object that the query returns is a ***ResultSet*** object. Use this method if the query could return one or more ***ResultSet*** objects. Retrieve the ***ResultSet*** objects returned from the query by repeatedly calling *Statement.getResultSet*.
- ***executeQuery***: Returns one ***ResultSet*** object.
- ***executeUpdate***: Returns an integer representing the number of rows affected by the SQL statement. Use this method if we are using INSERT,DELETE, or UPDATE SQL statements.

11.5.5. Processing a Query's *ResultSet*

An example of processing of Query **ResultSet** is shown in above. First of all it needs to obtains the metadata for the **ResultSet** as a **ResultSetMetaData** (package **java.sql**) object. The **metadata** describes the **ResultSet**'s contents. Programs can use metadata programmatically to obtain information about the **ResultSet**'s column names and types. **ResultSetMetaData** method **getColumnCount** are use to retrieve the number of columns in the **ResultSet**. First *for loop* is used to display the column names while second *for loop* display the data in each **ResultSet** row. First, the program positions the **ResultSet** cursor (which points to the row being processed) to the first row in the **ResultSet** with method **next**. Method **next** returns *boolean* value **true** if it's able to position to the next row; otherwise, the method returns **false**.

If there are rows in the **ResultSet**, second *for loop* extract and display the contents of each column in the current row. When a **ResultSet** is processed, each column can be extracted as a specific Java type. In fact, **ResultSetMetaData** method **getColumnType** returns a constant integer from class **Types** (package **java.sql**) indicating the type of a specified column. Programs can use these values in a *switch* statement to invoke **ResultSet** methods that return the column values as appropriate Java types. If the type of a column is **Types.INTEGER**, **ResultSet** method **getInt** returns the column value as an *int*. **ResultSet** *get* methods typically receive as an argument either a column number (as an *int*) or a column name (as a *String*) indicating which column's value to obtain.

For simplicity, this example treats each value as an *Object*. We retrieve each column value with **ResultSet** method **getObject** then print the *Object's String* representation. Unlike array indices, **ResultSet** column numbers start at 1. The *finally* block closes the **ResultSet**, the *Statement* and the database *Connection*. **NullPointerException** will throw if the **ResultSet**, *Statement* or *Connection* objects were not created properly.

We can also access the data in a **ResultSet** object through a cursor. This cursor is not a database cursor. This cursor is a pointer that points to one row of data in the **ResultSet** object. Initially, the cursor is positioned before the first row. We call various methods defined in the **ResultSet** object to move the cursor.

```
try {
    stmt = con.createStatement();
    ResultSet rs = stmt.executeQuery(query);
    while (rs.next()) {
        int ID = resultSet.getInt("AuthorID");
        String Fname = resultSet.getString("FirstName");
        String Lname = resultSet.getString("LastName");
        //System.out.println();
        System.out.println(ID + "\t\t" + Fname + "\t\t" + Lname);
    } // end while
} // end try
catch ( SQLException sqlException )
{
    sqlException.printStackTrace();
}
```